# PROGRAMARE ORIENTATĂ PE OBIECTE

şl.ing. Carmen ODUBĂŞTEANU

# Ce este un pachet ?

Pachet = Colecţie de clase şi interfeţe

Scopul:

- Organizarea lucrului
- Găsirea şi utilizarea mai uşoară a claselor
- Evitarea conflictelor de nume
- Controlul accesului

# Pachetele standard (J2SDK)

- **java.lang** - clasele de bază ale limbajului Java
- **java.io** - intrări/ieşiri, lucrul cu fişiere
- **java.util** - clase şi interfeţe utile
- **java.applet** - dezvoltarea de appleturi
- **java.awt** - interfaţa grafică cu utilizatorul
- **java.awt.event** - tratare evenimente
- **java.beans** - scrierea de componente reutilizabile
- **java.net** - programare de reţea
- **java.sql** - lucrul cu baze de date
- **java.rmi** - execuţie la distanţă
- **java.security** - mecanisme de securitate
- **java.math** - operaţii matematice cu numere mari
- **java.text** - lucrul cu texte, date şi numere independent de limbă
- **java.lang.reflect** - introspecţie
- **javax.swing** - interfaţa grafică cu utilizatorul, mult îmbogăţită faţă de AWT.

# Folosirea membrilor unui pachet

1. specificarea numelui complet:

numePachet.NumeClasa.

Button - numele scurt al clasei

java.awt - pachetul din care face parte

java.awt.Button - numele complet al clasei

Exemplu:

```
java.awt.Button b1 = new java.awt.Button("OK");
java.awt.Button b2 = new java.awt.Button("Cancel");
java.awt.TextField tf1 =
    new java.awt.TextField("Neplacut");
java.awt.TextField tf2 =
    new java.awt.TextField("Tot neplacut");
```

# Importul unei clase sau interfeţe

2. importul clasei respective

```
import numePachet.numeClasa;
```

```
//Pentru exemplul nostru:
import java.awt.Button;
import java.awt.TextField;
...
Button b1 = new Button("OK");
Button b2 = new Button("Cancel");
TextField tf1 = new TextField("Placut");
TextField tf2 = new TextField("Foarte placut");

//Problema:
import java.awt.Button;
import java.awt.TextField;
import java.awt.Rectangle;
import java.awt.Line;
import java.awt.Point;
import java.awt.Polygon;
```

# Importul la cerere

3. importul întregului pachet

import numePachet.*;

//Pentru exemplul nostru:

import java.awt.*;

...

Button b = new Button("OK");

Point p = new Point(0, 0);

import java.awt.C*; = eroare

Importate automat:

- pachetul java.lang
- pachetul curent
- pachetul implicit (fără nume)

# Ambiguităţi

```
import java.awt.*;
// Contine clasa List
import java.util.*;
// Contine interfata List
...
List x; //Declaratie ambigua
java.awt.List a = new java.awt.List(); //corect
java.util.List b = new ArrayList(); //corect
```

# Importul static

- Referirea constantelor statice ale unei clase:

```
import static numePachet.NumeClasa.*;
```

```
// Inainte de versiunea 1.5
import java.awt.BorderLayout.*;
...
fereastra.add(new Button(), BorderLayout.CENTER);

// Incepand cu versiunea 1.5
import java.awt.BorderLayout.*;
import static java.awt.BorderLayout.*;
...
fereastra.add(new Button(), CENTER);
```

# Crearea unui pachet

```
package numePachet;

//Fisierul Graf.java
package grafuri;
class Graf {...}
class GrafPerfect extends Graf {...}

//Fisierul Arbore.java
package grafuri;
class Arbore {...}
class ArboreBinar extends Arbore {...}
```

Pachetul implicit = directorul curent

# Organizarea fişierelor sursă

Organizarea surselor = foarte importantă

Recomandări:

- clasa - fişier
- sursa clasei C - fişierul C.java obligatoriu pentru clase publice
- pachet - director
- clasele pachetului - fişierele directorului

# Organizarea fişierelor sursă

/matematica
    /surse
        /geometrie
            /plan
                Poligon.java
                Cerc.java
            /spatiu
                Poliedru.java
                Sfera.java
        /algebra
            Grup.java
        /analiza
            Functie.java
            Matematica.java

# Organizarea fişierelor sursă

```
// Poligon.java
package geometrie.plan;
public class Poligon { . . . }

// Cerc.java
package geometrie.plan;
public class Cerc { . . . }

// Poliedru.java
package geometrie.spatiu;
public class Poliedru{ . . . }

// Sfera.java
package geometrie.spatiu;
public class Sfera { . . . }

// Grup.java
package algebra;
public class Grup { . . . }

// Functie.java
package analiza;
public class Functie { . . . }
```

# Tip abstract de date

- mulţime de date care au aceeaşi reprezentare şi pentru care este definit setul de operaţii care se pot executa asupra elementelor mulţimii respective.

- are două părţi:
  - o parte care defineşte reprezentarea datelor
  - o parte care defineşte operaţiile asupra datelor respective.

# Noţiunea de clasă

- O *clasă* defineşte un tip abstract de date.

Definiţie clasă:

*class nume{*

*lista_elementelor_membru*

*}*

Lista elementelor membru poate conţine:

- declaraţii de date;
- implementări de funcţii;
- prototipuri de funcţii abstracte.

- Datele declarate printr-o definiţie de clasă se numesc *date membru*
- Funcţiile definite sau pentru care este prezent numai prototipul în definiţia clasei, se numesc *funcţii membru* sau *metode.*
- Atât datele cât şi metodele pot avea modificatori de acces

# Modificatorii de acces

- ***Modificatorii de acces*** sunt cuvinte rezervate ce controlează accesul celorlalte clase la membrii unei clase. Specificatorii de acces pentru variabilele şi metodele unei clase sunt: public, protected, private şi cel implicit (la nivel de pachet).

| Specificator | Clasa | Subcls* | Pachet | Oriunde |
|---|---|---|---|---|
| **Private** | **X** | | | |
| **Implicit** | **X** | | **X** | |
| **Protected** | **X** | **X** | **X** | |
| **Public** | **X** | **X** | **X** | **X** |

***subclasă din alt pachet**

# Clasă

**Exemplu:**

```
class Complex {
        // date membru
    float real;
    float imag;
       // functii membru publice
    public void atribuire(float x, float y) {
          real = x;  imag=y;
   }
   public double retreal() {
          return real;
   }
   public void afiscomplex(){
          System.out.println(real+”+“+imag+”*i”);
   }
}
```

# Obiecte

- Un *obiect* este o dată de un tip definit printr-o clasă. Se spune că obiectul este o *instanţiere* a clasei respective.

- Formatul declaratiei unui obiect:

  *nume_clasă  nume_obiect;*

- Instanţierea obiectelor se face folosind operatorul *new.*

  *nume_obiect = new nume_clasă(..);*

# Obiecte

- Datele membru se alocă distinct la fiecare instanţiere a clasei. O excepţie o constituie datele membru care au clasa de memorare *static*, ea este o parte comună pentru toate instanţierile clasei şi  există într-un singur exemplar.

- Funcţiile membru sunt într-un singur exemplar oricâte instanţieri ar exista. Legătura dintre funcţii membru şi obiectul pentru care se face apelul se realizează folosind operatorul punct.

# Obiecte

- Exemplu de instanţieri pentru clasa *complex:*

  Complex z;

  z=new Complex();

  Sau:

  Complex z=new Complex();

- Atunci:

  z.atribuire(0,0);

  z.afiscomplex();  // afiscomplex(z); NUU

  – afişează numărul complex z (în cazul de faţă 0+0i).

  z.real

  z.imag

# Constructori

- Obiectele se generează şi se pot iniţializa la instanţiere cu ajutorul *constructorilor*
  - Funcţii membru ce au acelaşi nume cu numele clasei
  - Funcţii apelate automat la crearea obiectelor.
- Valorile de iniţializare se transferă constructorului şi ele joacă acelaşi rol ca parametrii efectivi de la apelurile funcţiilor obişnuite.
- Se pot defini mai mulţi constructori pentru o clasă. În acest caz ei au acelaşi nume, dar diferă prin numărul şi/sau tipurile parametrilor.

# Constructori

- Dacă există mai mulţi constructori, atunci la iniţializare se utilizează regulile de la apelurile funcţiilor supraîncărcate.

- Funcţiile constructor nu întorc valori, dar nu sunt precedaţi de cuvântul *void.*

- Dacă clasa nu conţine constructori, se generează  un constructor fără parametri, adică un *constructor implicit.* El are rolul numai de alocare a obiectelor clasei respective, fără a le iniţializa.

# Constructori

Exemplu:

```
class Complex {
    double real;
    double imag;
     public Complex (double x, double y)
            {real = x; imag = y;}
    public Complex ( )
            {real = 0; imag = 0;}
    …
 }
```

Exemple de instanţiere:

```
Complex z= new Complex();          // z = 0 + 0*i
Complex z1= new Complex(1,0);    // z1 = 1 + 0*i
```

# Realizarea încapsulării datelor

- accesul la datele sau funcţiile membre ale unei clase din orice punct al domeniului de definiţie al clasei s-ar putea rezolva simplu prin declararea de tip public a acestora
- o astfel de implementare nu respectă principiul încapsulării datelor şi se recomandă să fie evitată
- din punct de vedere al dreptului de acces la membrii clasei, o clasă bine definită permite încapsularea (sau ascunderea informaţiilor), prin care un obiect poate ascunde celor care-l folosesc modul de implementare, prin interzicerea accesului la datele şi funcţiile private sau protected.

# Realizarea încapsulării datelor

- în general, respectând principiul încapsulării, datele membre sunt declarate private sau protected şi nu pot fi accesate direct (pentru citire sau scriere) din funcţii nemembre ale clasei.
- Pentru citirea sau modificarea unora dintre datele membre protejate în clasa respectivă se pot prevedea funcţii membre de tip public, care pot fi apelate din orice punct al domeniului de definiţie al clasei şi fac parte din interfaţa clasei.
- De exemplu, pentru clasa Complex , o implementare care respectă principiul încapsulării, dar, în acelaşi timp permite accesul la datele private ale clasei poate arăta astfel:

# Realizarea încapsulării datelor

```
class Complex {
    private double real;
    private double imag;
     public Complex(double x, double y){
          real = x; imag = y;}
    public Complex ( ){
          real = 0; imag = 0;}
    public void set(double x, double y){
          real = x; imag = y; }
    public void setre(double x){
          real = x;}
    public void setim(double y){
          imag = y; }
    public double getre(){
          return real ;}
    public double getim() {
          return imag;}
```

# Realizarea încapsulării datelor

```
public void display(){
        System.out.println(real+”+“+ imag +”i”);
   }
   Complex add (Complex z){
        Complex rez=new Complex();
        rez.real=real+z.real;
        rez.imag=imag+z.imag;
        return rez;
   // return new Complex(real+z.real,
   imag+z.imag);
   }
}
```

# Realizarea încapsulării datelor

```
class test{
   public static void main(String arg[]){
        Complex c1=new Complex(), c2=new

        Complex(1,1);
        c1.set(7.2, 9.3);
        c1.display(); // afiseaza 7.2+9.3i
        c1.setre(1.3);
        c1.setim(2.8);
        c1.display(); // afiseaza 1.3+2.8i
        add(c1,c2); //gresit!!!
        Complex c3 = c1.add(c2);
/* System.out.println(c1.add(c2)); - doar daca
   redefiniti toString in clasa Complex
*/
   }
}
```

# Şiruri de caractere String, StringBuffer, StringTokenizer

Clase, metode şi exemple

# Şiruri de caractere

- char[]
- String

  String s = "abc";

  String s = new String("abc");

  char data[] = {’a’, ’b’, ’c’};

  String s = new String(data);
- StringBuffer

  Un şir de caractere constant (nu se doresc schimbări în porţiuni din conţinutul său pe parcursul execuţiei programului) va fi declarat de tipul String, altfel va fi declarat de tip StringBuffer. StringBuffer pune la dispoziţie metode pentru modificarea conţinutului şirului, cum ar fi: append, insert, delete, reverse.

# Şiruri de caractere

- Uzual, cea mai folosită modalitate de a lucra cu şiruri este prin intermediul clasei String.
- Clasa StringBuffer va fi utilizată predominant în aplicaţii dedicate procesării textelor cum ar fi editoarele de texte.

```
String s1="asd", s2="";
s1=s2+"a";
System.out.println(s1+" "+s2);
```

- Testarea egalităţii: equals

  ```
  if (nume.equals("duke")) { ... }
  ```

# Clasa String
# Crearea unui şir

- Constructorul implicit crează un şir vid:

  String s = new String();

- String str = "abc";

este echivalent cu:

  char data[] = {'a', 'b', 'c'};

  String str = new String(data);

OBS. Dacă elementele din vectorul data sunt modificate după crearea şirului str, atunci aceste modificări nu apar în acest şir.

- Construirea unui şir pe baza altui şir:

  String str2 = new String(str);

# Operaţii cu şiruri

- Metoda **length()** returnează lungimea unui şir

  Ex:

String s=“Hello”;

System.out.println(s.length());  // afişează 5

- Caracterele dintr-un şir pot fi accesate astfel:

  public char **charAt**(int index);

  Returnează caracterul din poziţia index. Domeniul de indexare este de la 0 la length() - 1.

  Ex. char ch; String s=“Hello”;

  ch = s.charAt(1); // ch = “e”

# Operaţii cu şiruri

- Concatenarea: +

  String s1 = "ab" + "cd";

  String s2 = s1 + 123 + "xyz“
- extrem de flexibil, permite concatenarea şirurilor cu obiecte de orice tip care au o reprezentare de tip şir de caractere.

- Exemple:

```
System.out.print("Vectorul v are" + v.length + "elem.");
String x = "a" + 1 + "b“;
```

- De fapt:

```
String x = new StringBuffer().append("a").append(1).
append("b").toString();
```

- Obs: şirul String s=1+2+"a"+1+2 va avea valoarea "3a12", primul + fiind operatorul matematic de adunare iar al doilea +, cel de concatenare a şirurilor.

# Operaţii cu şiruri

- **getChars() -** Copiază caracterele din şirul sursă în şirul destinaţie

  public void **getChars**(int srcBegin, int srcEnd, char[] dst, int dstBegin)

  - srcBegin – indexul primului caracter din sursă
  - srcEnd – indexul ultimului caracter din sursă
  - dst – vectorul destinaţie
  - dstBegin – poziţia de la care începe copierea în vectorul destinaţie
- **equals() –** Compară două şiruri la egalitate

  public boolean **equals**(String s2)

Ex: String s1,s2;

if (s1.equals(s2))…

- **equalsIgnoreCase()-** Compară două şiruri la egalitate fără să ţină cont de litere mici sau mari

  public boolean **equalsIgnoreCase**(String s2)

# Operaţii cu şiruri

- **startsWith()** – Testează dacă şirul începe cu prefixul specificat (la început sau dintr-o anumită poziţie)

  public boolean **startsWith**(String prefix)

  Ex.“Figure”.startsWith(“Fig”); // true

  public boolean **startsWith**(String prefix, int toffset)

  Ex.“figure”.startsWith(“gure”, 2); // true
- **endsWith() -** Testează dacă şirul se termină cu sufixul specificat

  public boolean **endsWith**(String suffix)

  Ex.

  String s=“Hellore”;

  s.endsWith(“re”); // true
- **compareTo()** – Compară două şiruri din punct de vedere lexicografic

  Rezultatul este
  - Negativ, dacă şirul precede şirul primit ca argument
  - Zero, dacă şirurile sunt egale
  - Pozitiv, dacă şirul urmează şirului primit ca argument

  public int **compareTo**(String anotherString)

  public int **compareToIgnoreCase**(String str)

# Operaţii cu şiruri

- **indexOf –** Caută prima apariţie a unui caracter sau şir într-un alt şir. Returnează -1 dacă nu îl găseşte.

  public int **indexOf**(int ch)

  public int **indexOf**(String str)

  Ex. String str = “How was your day today?”;
  str.indexOf(‘o’);  // 1
  str.indexOf(“was”); //4

- căutare începând de la o poziţie specificată:
  public int **indexOf**(int ch, int fromIndex)
  public int **indexOf**(String str, int fromIndex)

  Ex. String str = “How was your day today?”;
  str.indexOf(‘a’, 6); //14
  str.indexOf(“was”, 2); //4

- **lastIndexOf()** - Caută ultima apariţie a unui caracter sau şir într-un alt şir, similar lui indexOf.

# Operaţii cu şiruri

- **substring()** – Returnează un nou şir care este un subşir al şirului sursă.

  public String **substring**(int beginIndex)

  Ex: "unhappy".substring(2); // returnează "happy"

  public String **substring**(int beginIndex,
  int endIndex)

  Ex: "smiles".substring(1, 5); // returnează "mile“
- **concat()** – Concatenează  şirul specificat la şirul sursă

  public String **concat**(String str)

  "to".concat("get").concat("her"); // returnează "together"
- **replace()-** Returnează un nou şir în care toate apariţiile caracterului oldChar sunt înlocuite cu caracterul newChar.

  public String **replace**(char oldChar, char newChar)

  Ex. "mesquite in your cellar".replace('e', 'o'); returnează "mosquito in your collar"

# Operaţii cu şiruri

- **trim() -** Returnează şirul fără spaţiile albe de la început şi sfârşit

  public String trim()

  Ex. String s = “  Hi Mom!  “.trim();

  S este acum “Hi Mom!”

- **valueOf() –** Returnează reprezentarea sub formă de şir de caractere a argumentului

  public static String **valueOf**(char[] data)

  public static String **valueOf**(char c)

  public static String **valueOf**(boolean b)

  public static String **valueOf**(int i)

  public static String **valueOf**(long l)

  public static String **valueOf**(float f)

  public static String **valueOf**(double d)

# Operaţii cu şiruri

- **toLowerCase():** Converteşte toate caracterele la litere mici

  public String **toLowerCase**()

- **toUpperCase():** Converteşte toate caracterele la litere mari

  public String **toUpperCase**()

  Ex: “HELLO THERE”.toLowerCase();

  “hello there”.toUpperCase();

Exemplu:

String s=“HELLO THERE”;

s.toLowerCase(); // nu e suficient

s=s. toLowerCase(); // modifica sirul

# Operaţii cu şiruri

**public String[] split(String regex)**

- împarte sirul in subsiruri folosind un sir de delimitatori (expresie regulata)
- regex – expresia regulata de delimitare
- returneaza un vector de String-uri

Exemplu: pentru sirul "boo:and:foo“ avem urmatoarele rezultate:

| regex | rezultat |
|---|---|
| : | { "boo", "and", "foo" } |
| o | { "b", "", ":and:f" } |

**public String[] split(String regex, int limit)**

- Limit: de cate ori se aplica expresia regulata

# StringBuffer

- Este asemănător unui obiect String doar că este modificabil în sensul că există metode pentru modificarea lungimii şi a conţinutului său – append, insert.
- Crearea unui obiect StringBuffer:
  - StringBuffer()
  - StringBuffer(int size)
  - StringBuffer(String str)
- Principalele operaţii sunt adăugarea, inserarea şi ştergerea.
- Adăugarea:
  StringBuffer append(String str)
  StringBuffer append(int num)
- Inserarea se face într-o anumită poziţie.
  StringBuffer insert(int index, String str)
  StringBuffer append(int index, char ch)

# Operaţii cu StringBuffer

- **delete()** - şterge un subşir. Subşirul începe în poziţia specificată şi se termină în poziţia index de final - 1 sau până la sfârşitul şirului dacă nu se specifică poziţie de final.

public StringBuffer **delete**(int start, int end)

public StringBuffer **delete**(int start)

- **replace()** – Înlocuieşte un subşir din şirul sursă cu un alt şir

  public StringBuffer **replace**(int start, int end, String str)

- **substring()**

  public String substring(int start)

- **reverse()** – reversul şirului (ordine inversă a caracterelor)

  public StringBuffer **reverse**()

- **length()**

  public int **length**()

# Operaţii cu StringBuffer

- **capacity() –** Returnează capacitatea curentă a StringBuffer-ului. Capacitatea este dimensiunea disponibilă pentru adăugarea de noi elemente

  public int **capacity**()
- **charAt()**

  public char **charAt**(int index)
- **getChars()**

  public void **getChars**(int srcBegin, int srcEnd, char[] dst, int dstBegin)
- **setLength**() **–** Setează lungimea pentru obiectul de tip StringBuffer.

  public void **setLength**(int newLength)

# Exemple: StringBuffer

StringBuffer sb = new StringBuffer(“Hello”);

sb.length(); // 5

sb.capacity(); // 21 (16 caractere sunt adăugate dacă nu se specifică o dimensiune)

sb.charAt(1); // e

sb.setCharAt(1,’i’); // Hillo

sb.setLength(2); // Hi

sb.append(“l”).append(“l”); // Hill

sb.insert(0, “Big “); // Big Hill

sb.replace(3, 11, “”); // Big

sb.reverse(); // giB

# String / StringBuffer

String - Immutable

- Unui obiect de tip String nu i se poate modifica continutul! Se creaza un  nou obiect String modificat!

StringBuffer – Mutable

- Unui obiect de tip StringBuffer i se poate modifica continutul!

Exemplu:

```
String s= new String ("test");
StringBuffer sb= new StringBuffer ("test");
s.concat("unu");
sb.append("unu");
System.out.println(s);  // test
System.out.println(sb); // testunu
System.out.println(s.concat("unu") + " " + s);   // ?
```

# Clasa StringTokenizer

- Utilizată pentru împărţirea în atomi lexicali (tokens):

public **StringTokenizer**(String str, String delim)

public boolean **hasMoreTokens**()

public String **nextToken**()

Exemplu:

```
StringTokenizer st = new StringTokenizer("this, is a
    test.“,” ,.”);
while (st.hasMoreTokens()) {
     System.out.println( st.nextToken() );
}
```

Are următorul rezultat:

this

is

a

test

# Clasa Vector

De uitat in documentatia Java individual pentru laborator!!

```
Vector v=new Vector();
 System.out.println(v);
   // Afiseaza elementele vectorului v
```

Apeleaza implicit:

```
 System.out.println(v.toString());
```

Noi vom scrie doar :

**System.out.println(v);**